CO2 - AT1

Analytical Problem Solving.

Name : R.S. Kavinadhithya

Reg : 192511365

Course : Data Structures

Code : CSA0307

1. Supermarket Billing Counter :

* Normal Queue

Initial Queue

$C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4 \rightarrow C_5$

- C_1 is served
- C_2 is served
- A senior citizen arrives

In a normal queue [FIFO], the senior citizen joins at the end.

New Queue :

$C_3 \rightarrow C_4 \rightarrow C_5 \rightarrow$ Senior Citizen

The senior citizen must wait until all earlier customers are served

* Queue Based Solution with Priority

Use two queues :

→ Normal Queue :

Regular customers

→ Priority Queue → Senior citizens

Algorithm :

* Enqueue regular customers into the Normal Queue

* Enqueue Senior citizens into the Priority Queue

* while serving :

→ If the priority Queue is not empty, serve one senior citizen.

→ Then serve one customer from the Normal Queue.

→ Repeat.

Ex :

Initially :

* Normal Queue : $C_3 \rightarrow C_4 \rightarrow C_5$

* Priority Queue : Senior citizen

Serving order :

Senior citizen → $C_3 \rightarrow C_4 \rightarrow C_5$

This given priority to senior citizens while ensuring regular customer are not starve

Single linked list Reverse Using stack

Linked list :

S → I → M → A → T → S

```
#include < stdio.h >
#include < stdlib.h >
Struct node {
   char data;
   Struct node * next;
}
void printreverse ( Struct node * head)
{ char stack [10];
  int top =-1;
while (head != null) {
   Stack [++top] = head->data;
   head = head -> next;
}
while (top != -1) {
    print ("%c ", stack [top--]);
}
}
int main() {
   Struct node n1, n2, n3, n4, n5, n6;
```

Single linked list Reverse Using Stack.

Linked list :

S → I → M → A → T → S

```
#include < stdio.h >
#include < stdlib.h >

Struct node {
   char data;
   Struct node * next;
}
void printreverse ( struct node * head)
{
  char stack [10];
   int top =-1;
while (head != null) {
   Stack [++top] = head->data;
   head = head -> next;
}
while (top != -1) {
   print ("%c ", stack [top--]);
}
}
int main() {
  struct node n1, n2, n3, n4, n5, n6;
```

```
n1.data = 'S'; n1.next = &n2;
n2.data = 'I'; n2.next = &n3;
n3.data = 'M'; n3.next = &n4;
n4.data = 'A'; n4.next = &n5;
n5.data = 'T'; n5.next = &n6;
n6.data = 'S'; n6.next = Null;

Print f ("Reverse : ");
Print Reverse (&n1);
return 0;
}
```

output :

Reverse : S T A M I S